

## Lecture 4: Algorithmic Complexity

*Growth rates, asymptotic notation, and what they do and do not tell us*

Handout 4 of 11 · Department of Computer Science  
Lecture: Tuesday, 11:00 · Lab session: Thursday, 14:00

Prerequisite: Lecture 3 (arrays, lists, recursion)  
Assessed by: Coursework 1, question 2

---

These notes accompany Lecture 4 and are written to be read on their own, so the worked examples are given in full rather than left on the whiteboard. Everything here is examinable. The exercises at the end are not marked, but question 2 of Coursework 1 assumes you have attempted them.

### **Learning outcomes**

- State the definition of Big-O notation and apply it to a polynomial cost function.
- Order the common complexity classes by growth rate and predict how each responds to a change in input size.
- Distinguish best-case, average-case and worst-case analysis, and say which one a particular engineering decision depends on.
- Explain why an asymptotic classification does not by itself determine which of two algorithms runs faster on a given input.
- Distinguish auxiliary space from total space, and classify a sorting algorithm as in-place or out-of-place.

### **1. Why we measure complexity**

Two students submit programs that solve the same problem. On the same test file, one finishes in 0.8 seconds and the other in 2.1 seconds. It is tempting to conclude that the first student has the better algorithm. That conclusion is not supported by the measurement. What was measured is the behaviour of one implementation, written in one language, compiled with one set of options, running on one machine, against one input. Change any of those and the ranking can change.

The list of confounding factors is longer than it first appears. Processor clock speed and instruction set matter. So does the memory hierarchy: an algorithm whose working set fits in cache can beat one that does fewer operations but scatters them across memory. Compiler optimisation level matters, sometimes by a factor of three or more. Choice of language matters enormously; a careless program in a compiled language routinely outruns a carefully written one in an interpreted language by a factor of thirty to a hundred, which is easily enough to disguise a genuinely worse method. Background load on the machine adds noise. And the particular input chosen for the test may happen to suit one algorithm and not the other.

We want to compare the methods, not the circumstances. The standard approach is to abstract away from the machine entirely. Choose a measure of input size, normally written  $n$ : the number of elements in an array, the number of characters in a string, the number of vertices and edges in a graph. Choose an elementary operation that dominates the work, such as a comparison in a sorting routine or an array access in a search. Then count how many times that operation is performed, as a function of  $n$ , and call the result  $T(n)$ . Because  $T(n)$  is a count of operations and not a duration, it is the same whoever compiles the program and whatever hardware they run it

on.

There is a second, more practical reason to work this way. A stopwatch reading tells you about the input you happened to try. A cost function tells you what will happen when the input is ten times larger, or a thousand times larger. Systems are very rarely retired because they are too slow on today's data; they are retired because the data grew and the cost grew faster. Complexity analysis is the tool that lets you see that coming while the system still looks healthy.

A small example shows what this buys us. Suppose algorithm A performs  $100n$  elementary operations and algorithm B performs  $n^2$ . For  $n = 10$ , A does 1,000 operations and B does 100, so B is ten times cheaper. For  $n = 100$  they are level, at 10,000 operations each. For  $n = 1,000$ , A does 100,000 and B does 1,000,000, so A is now ten times cheaper, and the gap keeps widening: at  $n = 100,000$  A is a thousand times cheaper. Neither algorithm is simply "the fast one". Which is faster depends on  $n$ , and the two curves cross at a specific place. Keep this example in mind; we return to the crossing point in Section 5.

## 2. Big-O notation

Counting exact operations is still more detail than we usually want. The count  $3n^2 + 5n + 100$  and the count  $7n^2 + 2n + 4$  describe algorithms that behave the same way as  $n$  grows, even though the numbers differ. Big-O notation is the standard way of expressing that shared behaviour and discarding the rest.

**Definition.** Let  $f$  and  $g$  be functions from the positive integers to the non-negative reals. We say that  $f(n)$  is  $O(g(n))$  if there exist a constant  $c > 0$  and an integer  $n_0 \geq 1$  such that  $f(n) \leq c \cdot g(n)$  for every  $n \geq n_0$ .

In words: from some point onwards,  $f$  is bounded above by a constant multiple of  $g$ . Three features of that definition deserve to be read carefully, because each of them limits what a Big-O statement is entitled to claim.

**It is an upper bound, not an exact description.** The definition says  $f$  grows *no faster than*  $g$ . It does not say  $f$  grows as fast as  $g$ . A function that is  $O(n)$  is also, correctly,  $O(n^2)$  and  $O(2^n)$ , because a bound that is loose is still a bound. By convention we quote the tightest bound we can establish, which is why saying "binary search is  $O(n^2)$ " would be true but useless.

**It is asymptotic.** The definition constrains  $f$  only for  $n \geq n_0$ , and it does not say how large  $n_0$  is. Below that threshold the inequality is allowed to fail completely, and the notation makes no claim of any kind about what happens there. The constant  $c$  is similarly unconstrained; it may be 2, or it may be two million. Section 5 is about the consequences of these two silences.

**It describes growth rate, not speed.** This is the point most often lost. Big-O measures growth rate, not speed: it tells you how the running time scales as the input size increases, not how many seconds the algorithm takes on any particular machine. Two algorithms in the same Big-O class can differ in real running time by a factor of a hundred, because the constant factor that separates them is exactly what the notation throws away. Because the constant factor is discarded, an algorithm with a better Big-O classification is not guaranteed to be faster than one with a worse classification on any particular input; the classification only tells you which one must eventually

win as  $n$  grows without bound.

### Dropping constants and lower-order terms

In practice we do not work from the definition every time. We simplify a cost function by two rules: discard all constant multipliers, and discard every term except the fastest-growing one. So  $3n^2 + 5n + 100$  is  $O(n^2)$ . The 3 goes because constant multipliers are dropped; the  $5n$  and the 100 go because they are lower-order terms, dominated by  $n^2$ .

The justification is arithmetic. As  $n$  grows, the quadratic term takes over completely:

| $n$    | $3n^2$      | $5n$   | 100 | total $T(n)$ | share from $3n^2$ |
|--------|-------------|--------|-----|--------------|-------------------|
| 10     | 300         | 50     | 100 | 450          | 66.7%             |
| 100    | 30,000      | 500    | 100 | 30,600       | 98.0%             |
| 1,000  | 3,000,000   | 5,000  | 100 | 3,005,100    | 99.8%             |
| 10,000 | 300,000,000 | 50,000 | 100 | 300,050,100  | 99.98%            |

Table 1. Contribution of each term of  $T(n) = 3n^2 + 5n + 100$ . By  $n = 1,000$  the linear and constant terms together account for less than a fifth of one percent of the total.

To confirm it formally, we need only exhibit a  $c$  and an  $n_0$ . For every  $n \geq 1$  we have  $5n \leq 5n^2$  and  $100 \leq 100n^2$ , so  $3n^2 + 5n + 100 \leq 3n^2 + 5n^2 + 100n^2 = 108n^2$ . Taking  $c = 108$  and  $n_0 = 1$  satisfies the definition, and therefore  $T(n)$  is  $O(n^2)$ . A tighter pair also works: for  $n \geq 10$  we have  $5n \leq 0.5n^2$  and  $100 \leq n^2$ , giving  $T(n) \leq 4.5n^2$  with  $c = 4.5$  and  $n_0 = 10$ . Notice that both are valid. The definition asks for the existence of some  $c$  and  $n_0$ , not for the best ones, which is another reminder that the notation is deliberately coarse.

It is worth being explicit about why discarding the constants is legitimate, because the same step is what makes the result blunt. The constant multiplier reflects how much a single elementary step costs on a given machine with a given compiler and a given language, and those are precisely the factors Section 1 set out to eliminate. Removing them is what makes the classification portable. But the cost of that portability is that the classification retains no information at all about the regime in which those constants dominate. Big-O is designed to answer one question, namely how cost scales with input size, and it answers no other question.

### Related notation

Two companions to Big-O appear in the literature and occasionally in this module.  $\Omega(g(n))$  is the mirror image: an asymptotic *lower* bound, meaning  $f$  grows at least as fast as  $g$ .  $\Theta(g(n))$  means both at once, an asymptotically tight bound, and is the honest notation when we know the growth rate exactly rather than just bounding it above. Informally, most engineers say "O" where they mean " $\Theta$ "; the usage is so widespread that it is not worth fighting, but you should know the difference when you read a proof.

## 3. The growth-rate hierarchy

A small number of complexity classes account for the majority of algorithms you will meet. Learning their relative behaviour is more useful than memorising any individual result, because it lets you judge quickly whether a proposed approach is plausible at the scale you care about. The table below gives the approximate number of elementary operations each class implies.

| Class              | Name         | Typical example                          | n = 10 | n = 100                | n = 1,000           | n = 1,000,000         |
|--------------------|--------------|------------------------------------------|--------|------------------------|---------------------|-----------------------|
| O(1)               | constant     | array index; hash lookup                 | 1      | 1                      | 1                   | 1                     |
| O(log n)           | logarithmic  | binary search on sorted data             | 3      | 7                      | 10                  | 20                    |
| O(n)               | linear       | linear search; one pass over a list      | 10     | 100                    | 1,000               | 10 <sup>6</sup>       |
| O(n log n)         | linearithmic | mergesort; heapsort; quicksort (average) | 33     | 664                    | 9,966               | 2.0 × 10 <sup>7</sup> |
| O(n <sup>2</sup> ) | quadratic    | insertion sort; comparing all pairs      | 100    | 10,000                 | 10 <sup>6</sup>     | 10 <sup>12</sup>      |
| O(2 <sup>n</sup> ) | exponential  | enumerating all subsets                  | 1,024  | 1.3 × 10 <sup>30</sup> | ≈ 10 <sup>301</sup> | beyond astronomical   |

Table 2. Approximate operation counts by complexity class. Logarithms are base 2 and figures are rounded.

A useful way to internalise the table is to ask what happens when the input doubles. An O(1) algorithm is unaffected. An O(log n) algorithm performs one extra step. An O(n) algorithm does twice the work, and an O(n log n) algorithm slightly more than twice. An O(n<sup>2</sup>) algorithm does four times the work, so a tenfold increase in input becomes a hundredfold increase in cost. An O(2<sup>n</sup>) algorithm squares its own workload, which is why exponential algorithms hit a wall that hardware cannot move: a machine a thousand times faster buys you roughly ten additional elements of input, and then you are stuck again.

Two technical notes. First, the base of the logarithm never appears in the classification, because changing base multiplies by a constant ( $\log_2 n = \log_{10} n / \log_{10} 2$ ) and constants are dropped; O(log n) is unambiguous. Second, the classes are not equally spaced in practice. The step from O(n<sup>2</sup>) to O(n log n) usually decides whether a program is viable on large data, whereas the step from O(n) to O(n log n) is often invisible: for n = 1,000,000, log<sub>2</sub> n is only about 20, so the linearithmic algorithm does about twenty times the work of the linear one, a factor that a better implementation can easily recover.

## 4. Best case, average case and worst case

A single input size n does not determine the amount of work an algorithm does. Two arrays of the same length can send the same routine down very different paths, so a complete analysis reports three quantities: the fewest operations over all inputs of size n (best case), the most (worst case), and the expected number under some stated distribution of inputs (average case). The three are separate results and they are frequently in different complexity classes.

### Linear search

```
function LINEAR-SEARCH(A, target):
    for i = 0 to length(A) - 1:
        if A[i] == target:
            return i                # found it; stop immediately
    return NOT_FOUND               # fell off the end
```

The best case is one comparison, when the target sits at index 0: that is O(1). The worst case is n comparisons, when the target is in the last position or absent altogether: that is O(n). For the

average case we must state an assumption. If the target is present and equally likely to be at any index, the expected number of comparisons is  $(n + 1) / 2$ , which is still  $O(n)$  once the constant is dropped. Notice that the average-case result is only as trustworthy as the assumption behind it. If in your application the same few records are looked up constantly and the code moves recently found items to the front, the realistic average is far better than  $(n + 1) / 2$ , and the analysis above simply does not apply.

### Quicksort

```
function QUICKSORT(A, lo, hi):
    if lo >= hi:
        return                # zero or one element: already sorted
    p = PARTITION(A, lo, hi)   # choose a pivot, move smaller items left
    QUICKSORT(A, lo, p - 1)    # sort the left part
    QUICKSORT(A, p + 1, hi)    # sort the right part
```

Each call to PARTITION examines every element between lo and hi once, so the work at any one level of the recursion is proportional to the number of elements still being considered at that level, which is at most  $n$ . The total cost is therefore  $n$  multiplied by the depth of the recursion, and everything depends on how well the pivot splits the array.

When pivots land near the median, each call halves the range and the recursion is about  $\log n$  levels deep. That gives  $n \times \log n$  work overall, so quicksort is  $O(n \log n)$  in the average case, and random inputs are overwhelmingly likely to behave this way. When pivots land badly the picture changes completely. If the pivot is always the smallest or largest remaining element, each call removes just one item and the recursion is  $n$  levels deep, giving  $n + (n - 1) + (n - 2) + \dots + 1 = n(n + 1) / 2$  comparisons. So quicksort is  $O(n \log n)$  on average but  $O(n^2)$  in the worst case. The classic way to trigger the worst case is to hand a last-element-pivot implementation an array that is already sorted, which is an unfortunately common situation in real systems.

Practical implementations defend against this rather than accepting it. Choosing the pivot at random, or as the median of the first, middle and last elements, makes the degenerate case vanishingly unlikely for adversarially chosen data to hit by accident. Introsort goes further and monitors recursion depth, switching to heapsort if the depth exceeds roughly  $2 \log n$ , which converts quicksort's  $O(n^2)$  worst case into a hard  $O(n \log n)$  guarantee while keeping quicksort's speed on typical input.

Which of the three cases you should care about is an engineering question, not a mathematical one. A batch job that processes a million files overnight cares about the average, because the occasional bad input is amortised away. A flight control system or a request handler with a hard latency budget cares about the worst case, because an average is no defence when the deadline is missed. The best case is rarely decisive on its own, though it is worth knowing: insertion sort's  $O(n)$  best case on nearly-sorted data is the reason it appears in the next section at all.

## 5. Why asymptotic analysis can mislead on small inputs

Return to the definition in Section 2. It guarantees a bound only for  $n \geq n_0$ , and it allows the constant  $c$  to be arbitrarily large. Both of those allowances have practical consequences, and together they mean that a Big-O classification tells you nothing whatsoever about small inputs.

The constants that were dropped are not fictions. They stand for real work: the overhead of a function call, the cost of pushing and popping a stack frame, pivot selection, bounds checks, allocating a scratch buffer, and above all the memory access pattern, since a sequential sweep through a small array may sit entirely in L1 cache while a cleverer algorithm jumps around and misses. On a large input these costs are diluted across a great deal of productive work. On a small input they are essentially the whole cost.

Sorting is the standard illustration. Insertion sort is  $O(n^2)$ , which sounds disqualifying, but its constant factor is about as small as an algorithm's can be. Each step performs one comparison and one shift, it touches memory in a strictly sequential pattern, it makes no recursive calls, and it allocates nothing. It also has an  $O(n)$  best case, so on data that is already nearly ordered it barely does any work at all. Quicksort's per-call overhead is small but not zero, and on a twelve-element array you pay that overhead repeatedly in exchange for very little actual sorting.

An illustrative cost model makes the shape of the trade-off visible. Suppose insertion sort costs about  $0.5n^2$  units and quicksort about  $2n \log n + 30$  units, the trailing 30 standing for per-call setup that does not shrink with  $n$ :

| n     | insertion sort $\approx 0.5n^2$ | quicksort $\approx 2n \log n + 30$ | faster in this model          |
|-------|---------------------------------|------------------------------------|-------------------------------|
| 4     | 8                               | 46                                 | insertion sort, by about 6×   |
| 8     | 32                              | 78                                 | insertion sort, by about 2.4× |
| 16    | 128                             | 158                                | insertion sort, narrowly      |
| 20    | 200                             | 203                                | level pegging (the crossover) |
| 32    | 512                             | 350                                | quicksort                     |
| 100   | 5,000                           | 1,359                              | quicksort, by about 3.7×      |
| 1,000 | 500,000                         | 19,962                             | quicksort, by about 25×       |

Table 3. An illustrative model, not a measurement; the exact constants depend on the machine and the implementation. The shape, however, is real, and so is the location of the crossover.

Below the crossover the algorithm with the worse asymptotic complexity is genuinely the faster one, on the same data and the same machine. Above it the asymptotically better algorithm pulls ahead and never gives the lead back. Both halves of that sentence are ordinary consequences of the definition; neither is an anomaly.

**In practice.** This is not a theoretical curiosity: production sorting routines are built around it. Introsort, the hybrid used by the C++ standard library's `std::sort`, and Timsort, used by Python's `list.sort` and by Java's `sort` for object arrays, both stop subdividing once a partition or run falls below a fixed threshold of roughly 10 to 30 elements and finish that block with insertion sort, precisely because insertion sort's lower overhead beats quicksort's asymptotically better complexity on small arrays. Library authors treat this as routine engineering rather than as a special case.

The hybrid is easy to write, and you will implement one in the lab session:

```

SMALL = 16                                # tuned per library; typically 10 to 30

function INSERTION-SORT(A, lo, hi):
    for i = lo + 1 to hi:
        key = A[i]
        j = i - 1
        while j >= lo and A[j] > key:
            A[j + 1] = A[j]                # shift the larger element right
            j = j - 1
        A[j + 1] = key

function HYBRID-SORT(A, lo, hi):
    if hi - lo + 1 <= SMALL:
        INSERTION-SORT(A, lo, hi)        # cheaper here, despite being O(n^2)
        return
    p = PARTITION(A, lo, hi)
    HYBRID-SORT(A, lo, p - 1)
    HYBRID-SORT(A, p + 1, hi)

```

The same pattern appears well outside sorting. Strassen's matrix multiplication has a better exponent than the schoolbook algorithm, but its bookkeeping overhead means implementations only switch to it above matrix dimensions in the hundreds. A linear scan of a twenty-element array frequently beats a hash table lookup, because computing the hash and following a pointer costs more than twenty sequential comparisons in cache. Reading a small file into memory and sorting it beats building an index. In each case the asymptotically inferior method wins in a bounded region, and the region is often exactly where real workloads live.

The practical guidance follows directly. Use asymptotic analysis to choose the family of algorithms when inputs are large or unbounded, since nothing else predicts scaling. Do not use it to choose between candidates when the input is small or tightly bounded; there, measure on representative data and expect the constants to decide. And when the input size varies across the whole range, do what the standard libraries do and combine both, using the asymptotically good algorithm at the top and the low-overhead one underneath a threshold.

## 6. Space complexity

Everything so far has concerned time, but memory is analysed in exactly the same way and with the same notation. Space complexity expresses how the memory an algorithm requires grows with the input size  $n$ . The one distinction that must be kept clear is between two different things people mean by "the space it uses".

**Total space** counts everything, including the input itself. Since any algorithm that must read all of its input occupies at least  $O(n)$  total space, this measure rarely distinguishes one algorithm from another and is not usually what is quoted. **Auxiliary space** counts only the extra memory the algorithm allocates beyond the input: temporary arrays, bookkeeping structures, and the recursion stack. This is the interesting figure, and it is what is meant when a textbook says an algorithm "uses  $O(1)$  space". An algorithm whose auxiliary space is  $O(1)$ , or  $O(\log n)$  for the recursion stack alone, is called **in-place**; one that needs auxiliary space proportional to the input is **out-of-place**.



| Algorithm      | Time (average) | Time (worst)  | Auxiliary space                 | In-place? |
|----------------|----------------|---------------|---------------------------------|-----------|
| Insertion sort | $O(n^2)$       | $O(n^2)$      | $O(1)$                          | yes       |
| Heapsort       | $O(n \log n)$  | $O(n \log n)$ | $O(1)$                          | yes       |
| Quicksort      | $O(n \log n)$  | $O(n^2)$      | $O(\log n)$ stack, $O(n)$ worst | yes       |
| Mergesort      | $O(n \log n)$  | $O(n \log n)$ | $O(n)$                          | no        |
| Timsort        | $O(n \log n)$  | $O(n \log n)$ | $O(n)$                          | no        |

Table 4. Time and auxiliary space for the sorting algorithms discussed in this module.

Two rows in that table deserve comment. Mergesort is the canonical out-of-place sort: merging two sorted halves requires somewhere to put the result, so it needs a scratch buffer proportional to  $n$ . In exchange it offers an  $O(n \log n)$  guarantee in the worst case, not merely on average, and it is stable. Quicksort is called in-place because it rearranges elements within the original array, but it is not free: each pending recursive call occupies a stack frame, so balanced partitions cost  $O(\log n)$  auxiliary space and degenerate ones cost  $O(n)$ . Implementations reduce this by recursing into the smaller side first and looping on the larger, which caps the stack at  $O(\log n)$  regardless of how badly the pivots behave. Forgetting that the recursion stack is auxiliary space is one of the most common mistakes in this part of the module.

Time and space are frequently exchangeable, and choosing between them is a design decision rather than a matter of correctness. Memoising a recursive function trades  $O(n)$  memory for an often dramatic reduction in time; counting sort buys linear time by allocating an array the size of the key range; a database index accelerates queries at the cost of storage and slower writes. The trade is not always available on favourable terms. On an embedded controller with sixty-four kilobytes of RAM, or when the dataset only just fits in memory, auxiliary space is the binding constraint and an out-of-place algorithm is simply not an option however good its time bound looks.

Finally, note that space and time are not as independent as the two analyses suggest. Allocating a large auxiliary buffer costs time to allocate and to touch, and it evicts other data from cache, which slows down code that has nothing to do with the algorithm in question. This is one more channel through which the constant factors of Section 5 make themselves felt, and one more reason to measure a real implementation before declaring one approach better than another.

## Summary

- Timing measurements describe an implementation on a machine; complexity analysis describes the method, which is why we count operations as a function of input size  $n$ .
- $f(n)$  is  $O(g(n))$  when  $f(n) \leq c \cdot g(n)$  for all  $n$  beyond some  $n_0$ . It is an asymptotic upper bound on growth rate.
- Big-O describes how cost scales with input size, not how many seconds an algorithm takes. Constant factors and lower-order terms are dropped, so  $3n^2 + 5n + 100$  is  $O(n^2)$ .
- Best, average and worst case are separate results. Quicksort is  $O(n \log n)$  on average and  $O(n^2)$  in the worst case.



- Because the definition says nothing about inputs below  $n_0$ , an asymptotically worse algorithm can be the faster choice on small inputs, which is why standard libraries fall back to insertion sort below about 10 to 30 elements.
- Auxiliary space excludes the input and includes the recursion stack; it is what "in-place" refers to.

## Exercises

Not assessed, but recommended before the lab session. Solutions are discussed in Thursday's session rather than published.

- 1. Using the definition directly, show that  $7n + 2n \log n + 40$  is  $O(n \log n)$  by exhibiting a suitable  $c$  and  $n_0$ . Then find a smaller  $c$  that still works, and say what larger  $n_0$  it requires.
- 2. An algorithm performs  $T(n) = 6n^2 + 200n$  operations. For which values of  $n$  does the quadratic term exceed the linear term? Comment on what your answer implies about the range of  $n$  over which describing the algorithm as  $O(n^2)$  is informative.
- 3. Write down an input of length 8 that forces a quicksort implementation using the last element as pivot into its worst case, and count the comparisons. Propose two different changes that avoid it, and state what each costs.
- 4. A team replaces an  $O(n^2)$  routine with an  $O(n \log n)$  one and finds the program runs measurably slower in production. Give three explanations that are consistent with this handout, and describe an experiment that would distinguish between them.
- 5. You must sort 40 million records on a machine whose free memory is barely larger than the records themselves. Using Table 4, argue for one algorithm over the others, and state explicitly which column decided it.
- 6. Take the hybrid sort in Section 5 and set  $\text{SMALL} = 1$ . Explain in terms of the cost model in Table 3 what you would expect to happen to the running time on an array of 1,000 elements, and why.